Bezpieczeństwo haseł

Ćwiczenie pokazuje, co odróżnia hasło odporne na złamanie od hasła słabego. Punktem wyjścia jest plik ze skrótami kilkunastu haseł. Kolejne ataki narzędziem hashcat odczytują z tych skrótów coraz trudniejsze hasła: od słów z wycieków, przez słowa z typowymi podmianami znaków, po krótkie ciągi losowe. Dwa hasła w pliku nie zostają odczytane żadnym atakiem. Ćwiczenie dotyczy właśnie różnicy między tymi dwoma hasłami a pozostałymi.

Hasło nie jest przechowywane w bazie w postaci jawnej. Zamiast niego zapisuje się jego skrót (ang. *hash*): wynik funkcji, która z hasła wylicza ciąg o stałej długości i której nie da się wprost odwrócić. Przy logowaniu serwer liczy skrót z wpisanego hasła i porównuje go z zapisanym. Gdy napastnik wykradnie bazę, otrzymuje skróty, a nie hasła. Łamanie hasła polega na zgadywaniu: napastnik liczy skróty kolejnych kandydatów na hasło i szuka takiego, którego skrót zgadza się z wykradzionym. Na tym polega atak offline (prowadzony lokalnie, na skopiowanych skrótach, bez kontaktu z atakowanym serwerem).

Po zakończeniu ćwiczenia student potrafi:

- wyjaśnić, dlaczego serwer przechowuje skrót hasła, a nie hasło,
- rozpoznać funkcję skrótu po jej skrócie i wskazać właściwy tryb narzędzia hashcat,
- przeprowadzić atak słownikowy, atak z regułami podmian, atak kombinowany i atak maską,
- wyjaśnić, dlaczego słowo ze słownika oraz typowe podmiany znaków nie dają odporności,
- oszacować, jak długość i losowość hasła wpływają na czas potrzebny do jego złamania,
- wskazać różnicę między szybką funkcją skrótu a funkcją z rozciąganiem klucza (bcrypt) i podać, co składa się na bezpieczne hasło po stronie użytkownika i po stronie serwera.

Łamanie skrótów wolno prowadzić wyłącznie na skrótach własnych lub na pliku dołączonym do tego ćwiczenia. Skróty w pliku pochodzą z haseł ułożonych na potrzeby ćwiczenia. Kierowanie tych samych narzędzi na skróty pozyskane z cudzego systemu, bez pisemnej zgody właściciela, może stanowić przestępstwo. Ćwiczenie wykonuje się w całości na własnym komputerze.

Spis treści

0. Wymagania 3
1. Jak przechowuje się hasła 4
2. Narzędzie hashcat 5
3. Rozpoznanie funkcji skrótu 6
4. Atak słownikowy 7
5. Słownik z regułami podmian 8
6. Atak kombinowany 9
7. Atak maską 10
8. Funkcja bcrypt (opcjonalnie) 11
9. Co to znaczy bezpieczne hasło 12
10. Zapis obserwacji 14
11. Pytania kontrolne 15

0. Wymagania

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie:

- **hashcat w wersji 7 lub nowszej**, pobierany ze strony hashcat.net. hashcat to otwarte narzędzie do odzyskiwania haseł ze skrótów. W systemie Windows pobiera się je jako archiwum `7z`, które należy rozpakować; narzędzie nie wymaga instalacji. Po rozpakowaniu polecenie `hashcat.exe --version` z katalogu narzędzia wypisuje numer wersji. Polecenia w instrukcji zapisują nazwę narzędzia w skrócie jako `hashcat`. W systemie Windows, w programie PowerShell, uruchamia się je z katalogu narzędzia, poprzedzając nazwę przedrostkiem `.\`, czyli `.\hashcat.exe -m 1400 ...`.
- **Pliki ćwiczenia** z katalogu ćwiczenia: `hashes_sha256.txt` (skróty do złamania), `slownik.txt` (lista haseł do ataku słownikowego) i `reguly.rule` (reguły podmian). Część opcjonalną wykonuje się dodatkowo na pliku `hashes_bcrypt.txt`. Pliki te muszą być dostępne dla narzędzia: najprościej skopiować je do katalogu, z którego uruchamiany jest hashcat, albo w poleceniach podawać do nich pełne ścieżki.

Karta graficzna nie jest wymagana. hashcat działa także na samym procesorze; część podstawowa ćwiczenia (ataki na skróty SHA-256) jest na nim wykonalna w kilka minut. Część opcjonalna z funkcją bcrypt jest obliczeniowo cięższa i w rozsądnym czasie wymaga karty graficznej. Przy każdym kroku zaznaczono, czy wykonuje się go na procesorze, czy na karcie graficznej.

> **UWAGA**
>
> hashcat korzysta z procesora lub karty graficznej przez sterownik obliczeniowy. Na karcie NVIDIA wymaga sterownika CUDA, na karcie AMD sterownika z pakietu ROCm, a na procesorze środowiska OpenCL (w systemie Windows dostarcza je zwykle sterownik procesora). Listę wykrytych urządzeń wypisuje polecenie `hashcat.exe -I`. Jeśli hashcat nie znajdzie żadnego urządzenia, należy doinstalować sterownik zgodny z posiadanym sprzętem.

1. Jak przechowuje się hasła

Funkcja skrótu zamienia dane dowolnej długości na ciąg o stałej długości. Dla hasła `qwerty` funkcja SHA-256 daje zawsze ten sam wynik:

```
hasło:  qwerty
skrót:  65e84be33532fb784c48129675f9eff3a682b27168c0ea744b2cf58ee02337c5
```

W pliku skrótów zapisany jest sam skrót, bez hasła. Funkcja ma dwie własności istotne dla haseł. Jest jednokierunkowa: z samego skrótu nie da się wyliczyć hasła. Jest też powtarzalna: to samo hasło daje zawsze ten sam skrót. Dzięki pierwszej własności wyciek bazy nie ujawnia haseł wprost. Dzięki drugiej serwer może sprawdzić hasło przy logowaniu, nie przechowując go.

Powtarzalność jest jednak także słabością, którą wykorzystuje napastnik. Skoro to samo hasło daje zawsze ten sam skrót, napastnik może policzyć skróty wielu kandydatów na hasło i porównać je z wykradzionym. Gdy trafi na zgodność, zna hasło. Napastnik nie odwraca funkcji. Zgaduje hasło i liczy skrót w przód. Cały atak offline polega na powtarzaniu tych samych trzech czynności. Napastnik bierze kolejnego kandydata, liczy jego skrót i porównuje go ze skrótem wykradzionym.

O tym, jak szybko przebiega zgadywanie, decydują dwa czynniki: ilu kandydatów napastnik musi sprawdzić (czyli jakie jest hasło) oraz jak kosztowne jest policzenie jednego skrótu (czyli jaka to funkcja). Pierwsza część ćwiczenia zmienia hasła przy stałej, szybkiej funkcji SHA-256. Część opcjonalna zmienia funkcję na wolną (bcrypt) przy tych samych hasłach.

2. Narzędzie hashcat

hashcat przyjmuje plik ze skrótami i wytwarza kolejnych kandydatów na hasło, licząc dla każdego skrót i porównując go ze skrótami z pliku. Dwa przełączniki ustawiają najważniejsze parametry. Przełącznik `-m` wskazuje funkcję skrótu, a `-a` sposób wytwarzania kandydatów (tryb ataku).

Przełącznik	Znaczenie
`-m 1400`	funkcja skrótu SHA-256
`-m 3200`	funkcja skrótu bcrypt (część opcjonalna)
`-a 0`	atak słownikowy: kandydaci to kolejne wiersze słownika
`-a 1`	atak kombinowany: kandydaci to pary słów sklejone z dwóch słowników
`-a 3`	atak maską: kandydaci to wszystkie ciągi pasujące do wzorca
`-r PLIK`	zastosuj do słów ze słownika reguły podmian z pliku
`-i`	atak maską: zwiększaj długość kandydata po kolei
`--show`	wypisz dotąd złamane hasła

Numer funkcji dla `-m` zależy od tego, jaką funkcją policzono skrót. Bez niego hashcat nie rozpocznie pracy, bo nie wie, co liczyć. Jak ustalić właściwy numer, pokazuje następny rozdział.

hashcat zapisuje złamane hasła w pliku `hashcat.potfile` i pomija przy kolejnych uruchomieniach te już złamane. Dzięki temu po wykonaniu kilku ataków komplet wyników wypisuje jedno polecenie `--show`. Z tego samego powodu kolejne ataki w ćwiczeniu dokładają wyniki do poprzednich, zamiast zaczynać od zera. Przy powtórnym uruchomieniu tego samego ataku hashcat zgłasza, że skróty są już w pliku `hashcat.potfile`, i nie wypisuje haseł ponownie; komplet pokazuje wtedy polecenie `--show`.

3. Rozpoznanie funkcji skrótu

hashcat wymaga podania funkcji skrótu przełącznikiem `-m`. Bez niej nie rozpocznie pracy. W prawdziwym audycie rodzaj skrótu nie jest podany; trzeba go ustalić z samego skrótu. Jest to pierwszy krok pracy ze skrótami: zanim dobierze się atak, trzeba wiedzieć, jaką funkcję się łamie.

Część skrótów rozpoznaje się wprost z postaci. Długość i zestaw znaków zawężają możliwości: skrót złożony z 32 znaków szesnastkowych ma 128 bitów (tyle, co MD5), z 40 znaków 160 bitów (tyle, co SHA-1), a z 64 znaków 256 bitów (tyle, co SHA-256). Niektóre funkcje mają własny, rozpoznawalny zapis: skrót bcrypt zaczyna się od `$2a$`, `$2b$` lub `$2y$`, po czym następuje koszt i sól (losowa wartość dołączana do hasła).

Rozpoznanie ułatwiają narzędzia. hashcat ma wbudowany przełącznik `--identify`, który dla podanego skrótu wypisuje pasujące tryby `-m`.

> **KROK**
>
> Sprawdź, jakim funkcjom może odpowiadać skrót z pliku (wykonalne na procesorze):
>
> ```
> hashcat --identify hashes_sha256.txt
> ```

Dla skrótu o długości 64 znaków hashcat wypisuje kilkadziesiąt kandydatów, ponieważ tyle różnych funkcji daje skrót tej długości (lista skrócona):

```
      # | Name                                | Category
========+====================================+============
   1400 | SHA2-256                           | Raw Hash
  17400 | SHA3-256                           | Raw Hash
  17800 | Keccak-256                         | Raw Hash
  40087 | RIPEMD-256                         | Raw Hash
  40114 | HAVAL-256 (3 passes)               | Raw Hash
  40118 | BLAKE-256                          | Raw Hash
```

Sam skrót nie wskazuje więc jednoznacznie funkcji. Rozstrzyga kontekst. Do przechowywania haseł w aplikacjach używa się nielicznych, powszechnie znanych funkcji, a SHA-256 jest wśród nich najczęstszą funkcją dającą skrót tej długości. W tym ćwiczeniu skróty w pliku `hashes_sha256.txt` policzono funkcją SHA-256, czyli `-m 1400`; to jej numer podają polecenia w dalszej części.

Dla pliku `hashes_bcrypt.txt` rozpoznanie jest jednoznaczne. Prefiks `$2b$` jednoznacznie wskazuje bcrypt, a `--identify` podaje tryb `-m 3200`. Po prefiksie następuje koszt i sól, zapisane wprost w skrócie. Koszt jest wykładnikiem: wartość `$10$` oznacza 2 do potęgi 10, czyli 1024-krotne powtórzenie wewnętrznej operacji skrótu, opisanej w rozdziale o funkcji bcrypt. Każde zwiększenie kosztu o jeden podwaja czas liczenia skrótu.

> **UWAGA**
>
> Ten sam skrót rozpoznają też osobne narzędzia, na przykład Name-That-Hash (polecenie `nth`), które wypisuje kandydatów uszeregowanych według popularności. Żadne narzędzie nie zastępuje potwierdzenia rodzaju skrótu kontekstem, z którego skrót pochodzi.

4. Atak słownikowy

Najprostszy atak sprawdza jako kandydatów gotową listę haseł. Lista taka powstaje przede wszystkim z haseł ujawnionych we wcześniejszych wyciekach. Dołączony plik `slownik.txt` jest krótką listą tego rodzaju.

> **KROK**
>
> W katalogu narzędzia hashcat wykonaj atak słownikowy na pliku skrótów (wykonalne na procesorze):
>
> ```
> hashcat -m 1400 -a 0 hashes_sha256.txt slownik.txt
> ```

Pliki ćwiczenia muszą być dostępne dla narzędzia. Najprościej skopiować `hashes_sha256.txt`, `slownik.txt` i `reguly.rule` do katalogu, z którego uruchamiany jest hashcat, albo podać do nich pełne ścieżki. Po zakończeniu hashcat wypisuje podsumowanie; złamane hasła pojawiają się obok swoich skrótów:

```
65e84be33532fb784c48129675f9eff3a682b27168c0ea744b2cf58ee02337c5:qwerty
000c285457fc971f862a79b786476c78812c8897063c6fa9c045f579a3b2d63f:monkey
e4ad93ca07acb8d908a3aa41e920ea4f4ef4f26e7f86cf8291c5db289780a5ae:iloveyou
a9c43be948c5cabd56ef2bacffb77cdaa5eec49dd5eb0cc4129cf3eda5f0e74c:dragon
b068c5093aed09a10a9732de95a450c094e59f0ac8afede78474b8c75eda95b3:veritas

Recovered........: 5/14 (35.71%) Digests
```

Atak odczytał pięć haseł. Wszystkie są słowami z listy: zwykłymi słowami (`monkey`, `dragon`), hasłem z czołówki wycieków (`qwerty`) lub krótką frazą (`iloveyou`). Trafienie jest natychmiastowe, bo wystarczyło sprawdzić 167 kandydatów ze słownika.

Wśród złamanych jest `veritas`, łacińskie słowo. Wybór rzadkiego słowa nie daje odporności: łacina, tak jak każdy inny język, ma swoje słowniki, a listy słów powstają też ze zrzutów encyklopedii i innych tekstów. Słowo pozostaje słowem na jakiejś liście, niezależnie od tego, jak rzadkie się wydaje. Pozostałe skróty nie zostały złamane, ponieważ odpowiadających im haseł nie ma w słowniku.

5. Słownik z regułami podmian

Napastnik wie, że użytkownicy zmieniają słowa w przewidywalny sposób: zapisują pierwszą literę wielką, dopisują na końcu cyfry albo rok, zastępują litery podobnymi znakami (`a` na `@` lub `4`, `e` na `3`, `o` na `0`, `s` na `$`). Zapis `P@ssw0rd` zamiast `password` to właśnie taka podmiana, zwana leet. hashcat stosuje te zmiany automatycznie: reguła to przepis na przekształcenie słowa, a plik reguł jest listą takich przepisów. Przy ataku z regułami każdy kandydat ze słownika zostaje sprawdzony we wszystkich wariantach z pliku.

Dołączony plik `reguly.rule` zawiera typowe przekształcenia. Kilka przykładów jego wierszy:

Reguła	Znaczenie
`c`	zapisz pierwszą literę wielką, resztę małymi
`c $1 $2 $3`	pierwsza litera wielka, dopisz na końcu `123`
`c $2 $0 $2 $4`	pierwsza litera wielka, dopisz na końcu `2024`
`sa4 se3 si! so0 ss$`	zamień `a` na `4`, `e` na `3`, `i` na `!`, `o` na `0`, `s` na `$`

KROK

Powtórz atak słownikowy, dokładając reguły podmian (wykonalne na procesorze):

```
hashcat -m 1400 -a 0 hashes_sha256.txt slownik.txt -r reguly.rule
```

Tym razem hashcat odczytuje dziewięć haseł: pięć poprzednich i cztery nowe. Komplet dotychczas złamanych haseł wypisze polecenie `hashcat -m 1400 hashes_sha256.txt --show`. Cztery nowe, dopisane przez reguły, to:

```
51c235b349dd4af59b9f2ae219cae37263dcf084599cb8537beec0cf19d8b82b:P4$$w0rd
57458189063b63404f6f98952adeda2eda78ae33b4154e2812d28144ffc06fcf:Z4j3c!4
83d738dd35b12c292280e6280dab153cf464a432cbba791be0c911804f5a5b31:Kasia123
e7cb19ce346eb9de61867ba40c0f4a51ebf2af4282bdd6c27247df842c562562:Wiosna2024
```

Każde z tych haseł powstało z jednego słowa ze słownika przez przekształcenie z pliku reguł. `Kasia123` to słowo `kasia` z pierwszą literą wielką i dopisanym `123`. `Wiosna2024` to `wiosna` z dopisanym rokiem. `P4$$w0rd` i `Z4j3c!4` to `password` i `zajecia` po podmianie leet. Podmiana liter i dopisana cyfra nie dają odporności, ponieważ napastnik zna te wzorce i każe narzędziu je odtworzyć. Odporność dałoby dopiero porzucenie słowa jako podstawy hasła.

6. Atak kombinowany

Hasło można też zbudować z dwóch słów. Napastnik odpowiada atakiem kombinowanym: sprawdza jako kandydatów wszystkie pary słów sklejone ze słownika (`-a 1` z tym samym słownikiem podanym dwa razy).

> **KROK**
>
> Wykonaj atak kombinowany (wykonalne na procesorze):
>
> ```
> hashcat -m 1400 -a 1 hashes_sha256.txt slownik.txt slownik.txt
> ```

Atak odczytuje hasło `wiosnalato`, złożone ze słów `wiosna` i `lato`:

```
eb2b22a449ae9b48888dfdcc474b1b6259f14141169e4db695bac490b947f4ea:wiosnalato
```

Sklejenie dwóch krótkich, pospolitych słów nie zwiększa istotnie liczby kandydatów. Przy słowniku liczącym 167 słów wszystkich par jest około dwudziestu ośmiu tysięcy, co narzędzie sprawdza natychmiast. Hasło z wielu słów ma sens dopiero wtedy, gdy słowa są liczne, długie i dobrane losowo, a nie są dwoma wyrazami pasującymi do siebie znaczeniem.

7. Atak maską

Hasło bez słów, złożone z przypadkowych znaków, nie padnie ani w ataku słownikowym, ani z regułami. Pozostaje atak maską: sprawdzenie po kolei wszystkich ciągów pasujących do wzorca. Maska opisuje, z jakich znaków i jak długie są kandydaci. Oznaczenie `?l` to dowolna mała litera, `?d` to dowolna cyfra; `-1 ?l?d` tworzy zestaw znaków złożony z małych liter i cyfr, a `?1` oznacza znak z tego zestawu. Przełącznik `-i` każe zwiększać długość po kolei, od jednego znaku w górę.

> **KROK**
>
> Wykonaj atak maską na ciągach małych liter i cyfr, o długości do siedmiu znaków (długość do sześciu znaków jest wykonalna na procesorze w kilka minut; siódmy znak jest wyraźnie szybszy na karcie graficznej):
>
> ```
> hashcat -m 1400 -a 3 hashes_sha256.txt -1 ?l?d -i --increment-min 1 --increment-max 7 ?1?1?1?1?1?1?1
> ```

Atak odczytuje dwa hasła: `k7p2mq` (sześć znaków) oraz `x9f3azq` (siedem znaków):

```
73037149cf9a951efd784abae3a8c3828e010c2b54cbee0a1f6b71d8a0391efd:k7p2mq
5f048ef93a299ab826f8d8b14b1a8bbbd7bc430f68aa71aabfd34e21dd10255c:x9f3azq
```

W czasie pracy hashcat pokazuje ekran stanu. Pole `Guess.Mask` to bieżąca długość kandydata, `Progress` udział sprawdzonych kandydatów, `Recovered` liczba odczytanych skrótów, a `Speed` prędkość, która zależy od sprzętu:

```
Session..........: hashcat
Status...........: Running
Hash.Mode........: 1400 (SHA2-256)
Guess.Mask.......: ?1?1?1?1?1?1?1 [7]
Guess.Charset....: -1 ?l?d
Speed.#01........: 8603.7 MH/s
Recovered........: 11/14 (78.57%) Digests
Progress.........: 42216710144/78364164096 (53.87%)
```

Atak maską sprawdza całą przestrzeń kandydatów, więc jego czas rośnie z długością hasła. Zestaw `?l?d` ma 36 znaków, zatem haseł o danej długości jest tyle, co 36 do tej potęgi. Każdy dołożony znak mnoży liczbę kandydatów, a przez to i czas, przez 36:

Długość	Liczba kandydatów	Procesor laptopa (ok. 100 MH/s)	Karta GTX 1650 (ok. 3 GH/s)
6 znaków	2,2 mld	ok. 20 s	ok. 1 s
7 znaków	78 mld	ok. 13 min	ok. 26 s
8 znaków	2,8 bln	ok. 8 godz.	ok. 15 min
10 znaków	3,7 biliarda	ok. rok	ok. dwa tygodnie

Wartości w tabeli są rzędami wielkości; dokładny czas zależy od sprzętu i wybranego zestawu znaków. Wniosek nie zależy od sprzętu: krótkie hasło losowe pada, bo jego przestrzeń jest mała. O odporności decyduje długość. Każdy dołożony znak przedłuża atak wielokrotnie, więc hasło dostatecznie długie jest poza zasięgiem ataku maską niezależnie od użytego sprzętu. Z tego samego powodu zawężenie zestawu znaków (same małe litery, sam alfabet jednego języka) skraca atak, a rozszerzenie go (wielkie litery, cyfry, znaki specjalne) wydłuża.

8. Funkcja bcrypt (opcjonalnie)

Dotąd funkcja skrótu pozostawała stała (SHA-256), a zmieniały się hasła. W części opcjonalnej jest przeciwnie. Hasła są te same, ale zapisane inną funkcją. Plik `hashes_bcrypt.txt` zawiera skróty tych samych czternastu haseł policzone funkcją bcrypt (`-m 3200`).

SHA-256 projektowano tak, aby liczyć się szybko. Dla haseł jest to wada: skoro serwer liczy skrót szybko, napastnik także. bcrypt projektowano odwrotnie. Stosuje rozciąganie klucza (ang. *key stretching*): tę samą operację powtarza wielokrotnie, przez co policzenie jednego skrótu jest celowo wolne. Serwer liczy go raz przy logowaniu i opóźnienia nie zauważa; napastnik, który musi policzyć miliardy skrótów, odczuwa je w pełni. bcrypt dokłada też do każdego hasła sól (losową wartość zapisaną obok skrótu), więc ten sam skrót trzeba łamać osobno dla każdego konta.

> **KROK**
>
> Powtórz atak słownikowy i atak z regułami na pliku bcrypt (wymaga karty graficznej, aby zakończyć się szybko):
>
> ```
> hashcat -m 3200 -a 0 hashes_bcrypt.txt slownik.txt
> hashcat -m 3200 -a 0 hashes_bcrypt.txt slownik.txt -r reguly.rule
> ```

Ataki odczytują te same słabe hasła co na SHA-256. Zmienia się tylko czas liczenia skrótu. Na laptopie bez wydajnej karty graficznej SHA-256 sprawdza około stu milionów kandydatów na sekundę, a bcrypt o koszcie 10 tylko kilkaset:

```
-m 1400 (SHA-256)   Speed: ~100 MH/s
-m 3200 (bcrypt)    Speed:  ~300  H/s
```

Różnica jest rzędu kilkuset tysięcy razy. Dla ataku słownikowego nie ma ona znaczenia. Narzędzie sprawdzi 167 kandydatów i tak w ułamku sekundy, więc słowo ze słownika pada na bcrypcie równie pewnie jak na SHA-256. Pierwszy wniosek jest taki, że rozciąganie klucza nie ratuje słabego hasła. Słowo ze słownika to wciąż jedno zgadnięcie, niezależnie od tego, jak wolno liczy się skrót.

Wniosek drugi dotyczy ataku maską. Przy prędkości kilkuset skrótów na sekundę sprawdzenie 2,2 miliarda kandydatów (hasło sześcioznakowe) zajęłoby na laptopie miesiące, a hasło siedmioznakowe lata. To samo, co na SHA-256 zajmuje sekundy lub minuty, na bcrypcie staje się niewykonalne. Rozciąganie klucza nie pomaga przy słabym haśle, ale skutecznie zamyka atak siłowy na hasło losowe. Dlatego do przechowywania haseł używa się funkcji takich jak bcrypt, scrypt lub Argon2, a nie SHA-256 ani MD5.

9. Co to znaczy bezpieczne hasło

W pliku zostały dwa nieodczytane skróty. Odpowiadają im hasła `xK9#mL2$vQ8pR4wZ` oraz `7bT!gW4^nD9&qF2s`. Nie padły w żadnym ataku, bo łączą wszystkie cechy, których brakowało pozostałym: nie są słowem ani parą słów, więc atak słownikowy i kombinowany ich nie obejmują; nie są słowem po podmianie, więc reguły nic nie dają; a przy długości szesnastu znaków i pełnym zestawie znaków ich przestrzeń jest tak duża, że atak maską nie kończy się w realnym czasie na żadnym sprzęcie.

Stąd wynika, co składa się na bezpieczne hasło:

- **Długość** decyduje najbardziej. Każdy dołożony znak mnoży liczbę kandydatów, więc kilka znaków więcej przenosi hasło z zasięgu ataku maską poza niego.
- **Losowość** jest konieczna, bo atak słownikowy i reguły omijają długość, sięgając wprost po słowa i ich typowe warianty. Hasło musi nie dać się odgadnąć ze słów, imion, dat i podmian.
- **Brak znaczenia.** Słowo, fraza, imię z rokiem czy para pasujących słów to podstawy, które narzędzia sprawdzają najpierw. Bezpieczne hasło nie opiera się na niczym, co da się wyliczyć ze słownika.

Liczbę kandydatów, których napastnik musi sprawdzić, wygodnie mierzy się entropią (miarą nieprzewidywalności w bitach): hasło o entropii n bitów ma tyle możliwych postaci, co 2 do potęgi n. Każdy dołożony bit podwaja liczbę kandydatów. Długość i losowość podnoszą entropię, a słowo ze słownika i typowe podmiany ją obniżają, bo zawężają zbiór haseł, które napastnik musi rozważyć.

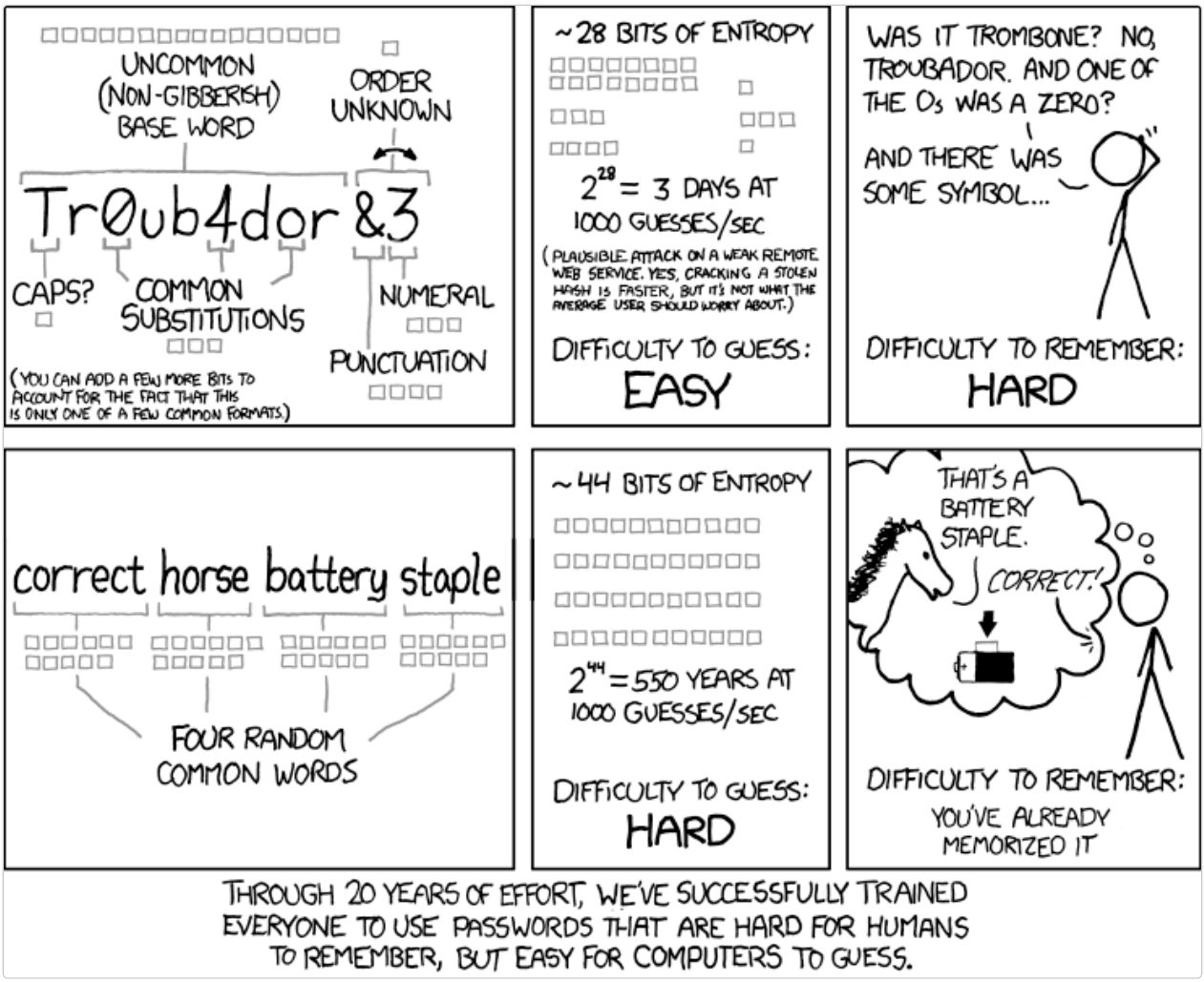
Dla hasła dobranego losowo entropię liczy się jako liczbę znaków pomnożoną przez logarytm dwójkowy z liczby znaków, spośród których losowany jest każdy znak:

$$\text{entropia} = \text{długość} \cdot \log_2(\text{liczba znaków}) \tag{1}$$

Dla zestawu `?l?d` z ataku maską (36 znaków) jeden znak wnosi `log2 36`, czyli około 5,2 bita (równanie 1), więc hasło sześcioznakowe ma nieco ponad 31 bitów. Odpowiada to przestrzeni ataku maską (około 2,2 miliarda kandydatów), bo 2 do potęgi 31 to mniej więcej tyle. Hasło złożone z losowych słów liczy się tak samo, biorąc liczbę słów i liczbę słów w zbiorze, z którego się je losuje: cztery słowa z listy dwóch tysięcy wnoszą po `log2 2000`, czyli po około 11 bitów, razem około 44 bitów.

Wzór obowiązuje tylko wtedy, gdy każdy znak lub słowo jest dobrany naprawdę losowo. Dla hasła układanego przez człowieka rzeczywista entropia jest znacznie niższa, niż wynikałoby z długości i zestawu znaków. `P4$$w0rd` ma osiem znaków z liter, cyfr i znaków specjalnych, ale powstało z jednego słownikowego słowa przez przewidywalną podmianę, więc zbiór realnych kandydatów jest mały. Dlatego pada w kilka chwil, mimo pozornie wysokiej złożoności. Entropia policzona z długości i zestawu znaków to górna granica, osiągana jedynie przy doborze losowym.

W tych samych kategoriach bezpieczeństwo haseł ujmuje znany komiks xkcd (rys. 1). Zestawia on hasło zbudowane ze słowa po podmianach (`Tr0ub4dor&3`, około 28 bitów entropii) z hasłem złożonym z czterech losowych słów (`correct horse battery staple`, około 44 bitów). To drugie jest trudniejsze do złamania, a zarazem łatwiejsze do zapamiętania.



Rys. 1. Siła hasła według entropii. Hasło oparte na jednym słowie z podmianami ma niską entropię mimo pozornej złożoności; cztery losowe słowa dają entropię wyższą. Podane tempo (1000 prób na sekundę) dotyczy zgadywania przez zdalną usługę; atak offline na wykradziony skrót szybkiej funkcji jest znacznie szybszy, ale porównanie entropii pozostaje w mocy. Źródło: Randall Munroe, xkcd.com/936, licencja CC BY-NC 2.5.

Komiks nie przeczy wynikowi ataku kombinowanego. Dwa pospolite, pasujące do siebie słowa (jak `wiosnalato`) to nadal niska entropia i padają szybko. Dopiero wiele słów dobranych losowo z dużego zbioru daje entropię porównywalną z długim hasłem losowym.

Hasła spełniającego te warunki nie da się zapamiętać dla każdego konta osobno. Służy do tego menedżer haseł: program, który generuje losowe hasła i przechowuje je zaszyfrowane, chronione jednym hasłem głównym. Użytkownik zapamiętuje tylko hasło główne, a dla każdego konta ma inne, długie, losowe hasło. Osobne hasło dla każdego konta jest ważne dlatego, że po wycieku z jednego serwisu napastnik próbuje tego samego hasła w innych; różne hasła zamykają tę drogę.

Hasło jest jednym składnikiem. Drugim jest uwierzytelnianie dwuskładnikowe (ang. *two-factor authentication*), w którym do hasła dochodzi kod z aplikacji lub klucz sprzętowy. Nawet poznane hasło nie wystarcza wtedy do zalogowania. Mechanizmy te omawia osobny wykład; tutaj istotne jest, że samo hasło, choćby mocne, nie jest jedyną linią obrony.

Po stronie serwera obrona jest komplementarna. Programista nie wpływa na to, jakie hasła wybierają użytkownicy, ale wybiera funkcję skrótu. Przechowanie haseł wolną, soloną funkcją (bcrypt, scrypt, Argon2) sprawia, że nawet po wycieku bazy łamanie jest kosztowne, a atak maską na mocne hasła niewykonalny. Żaden z tych środków nie wystarcza osobno: słabe hasło padnie mimo wolnej funkcji, a mocne hasło zapisane szybką funkcją bez soli jest narażone na masowe łamanie po wycieku.

10. Zapis obserwacji

Wynik ćwiczenia warto zapisać w formie, w jakiej opisuje się słabe hasła w raporcie z audytu: które hasła padły, jakim nakładem i dlaczego. Tabela poniżej jest wzorem; można ją uzupełnić o własne pomiary czasu.

Atak	Odczytane hasła	Wspólna cecha
słownikowy	`qwerty`, `monkey`, `iloveyou`, `dragon`, `veritas`	słowo z listy
słownik z regułami	`P4$$w0rd`, `Z4j3c!4`, `Kasia123`, `Wiosna2024`	słowo po typowej podmianie
kombinowany	`wiosnalato`	dwa pospolite słowa
maską	`k7p2mq`, `x9f3azq`	krótki ciąg losowy
nieodczytane	`xK9#mL2$vQ8pR4wZ`, `7bT!gW4^nD9&qF2s`	długie i losowe

Opis słabego hasła w raporcie zawiera: gdzie wystąpiło, na czym polega słabość (słowo, podmiana, zbyt krótki ciąg), jaki atak je odczytuje i w jakim czasie, oraz zalecenie. Zalecenie po stronie użytkownika to hasło długie i losowe z menedżera haseł; po stronie serwera to przechowanie haseł wolną, soloną funkcją i wymuszenie minimalnej długości hasła.

11. Pytania kontrolne

1. Dlaczego serwer przechowuje skrót hasła, a nie samo hasło? Co napastnik otrzymuje po wycieku bazy?
2. Na czym polega atak offline na skrót? Dlaczego powtarzalność funkcji skrótu jest w nim konieczna?
3. Dlaczego rzadkie słowo (`veritas`) pada w ataku słownikowym tak samo jak słowo pospolite?
4. Dlaczego podmiana liter (`P@ssw0rd`) i dopisany rok (`Wiosna2024`) nie dają odporności?
5. Jak długość hasła wpływa na czas ataku maską? Dlaczego dołożenie jednego znaku zmienia go wielokrotnie?
6. Dlaczego funkcja bcrypt nie ratuje słabego hasła, a jednocześnie zamyka atak maską na hasło losowe?
7. Co składa się na bezpieczne hasło po stronie użytkownika, a co po stronie serwera? Dlaczego żaden z tych środków nie wystarcza osobno?

> **DALSZA PRAKTYKA**
>
> Dokumentacja trybów ataku narzędzia hashcat jest dostępna pod adresem hashcat.net/wiki. Własne hasła można sprawdzić, licząc ich skrót i próbując je złamać opisanymi atakami. Łamanie wolno prowadzić wyłącznie na skrótach własnych lub za pisemną zgodą właściciela systemu.